

PROJECT DOCUMENT

Blood Supply Coordination Platform

One live picture of blood stock across hospitals, blood banks and volunteer donors, so units are used before they expire and shortages are filled early.

DATE	Sep 30, 2026
------	--------------

AUTHOR	@ADIL
--------	-------

COVERS	Overview · problem · goals and scope · roles · modules · requirements · technology stack
--------	--

STATUS	Built and evaluated on simulated data
--------	---------------------------------------

CONTENTS

What this document covers

The platform, the problem behind it, what it will and will not do, who uses it, and how it is built.

01 Overview

02 Problem

03 Goals, scope and non-goals

04 Users and roles

05 What the system does

06 Functional requirements

07 Non-functional requirements

08 Technology stack and why

SECTION 1

Overview

The Blood Supply Coordination Platform gives hospitals, blood banks and volunteer donors one live picture of blood stock, so units are used before they expire and shortages are filled early.

It has three parts: an **inventory network** across sites, a **forecasting and redistribution engine**, and a **verified donor network** for requests that stock cannot cover.

WORKED EXAMPLE

Hospital A has too much. Hospital B has too little.

Hospital A holds **12 units of A+ red cells** that expire in 4 days and are rarely used. Hospital B, 20 km away, has a cardiac surgery list tomorrow and only **3 units** left. Today neither hospital knows about the other. With the platform, the system proposes moving **8 units from A to B**; if the network still cannot cover a request, it invites eligible donors nearby.

Decision support only

The platform is a coordination and decision-support layer. It does not replace licensed blood bank systems, and it is built and evaluated on simulated data.

Problem

Blood is perishable and type-specific, yet it is usually managed one hospital at a time, so one area can discard units while another rations them.

The pressure is real

On 27 July 2026 the American Red Cross declared its second-ever national blood supply crisis after donations fell to a four-year summer low. It reported less than a one-day national supply of type O positive and began limiting type O distribution to hospitals.

Shortage is not only a donor problem. A 2026 survey of transfusion service staff in Riyadh (64 people directly managing shortages) describes it as a system-level challenge. The barriers they named most were limited donor availability, high workload, insufficient staffing, high transfusion demand and limited decision-making authority. The author also notes that hospital-based supply models can limit efficient redistribution of blood between sites.

The platform targets the operational gaps in that picture:

GAP TODAY	EFFECT	PLATFORM RESPONSE
Stock is visible only inside each hospital	Near-expiry units are discarded while other sites run short	Shared live inventory with expiry-aware transfers
Demand is judged by habit	Over-ordering, or emergency requests at short notice	Forecasts per site, component and blood group
Donors are found by phone calls and social posts	Slow, unverified, and the same few donors are asked repeatedly	Verified, request-based donor matching
Temporarily deferred donors are not followed up	Eligible donors are lost	Eligibility tracking and return reminders
No shared measure of what works	Policies cannot be tested	Simulator and standard metrics

SECTION 3

Goals, scope and non-goals

The project aims to show, in simulation, that shared inventory plus forecast-driven redistribution wastes fewer units and causes fewer shortages than sites working alone.

Goals

- Cut expired units and stockouts compared with independent site management.
- Give every role a single, live, trustworthy view of stock, requests and donors.
- Fill requests that stock cannot cover through verified, consent-based donor matching.
- Protect donor privacy by design, with no public directory of personal details.
- Produce measurable results from repeatable simulation experiments.

In scope

- Multi-site inventory tracking for red cells, platelets and plasma.
- Demand forecasting and transfer recommendations.
- Donor registration, eligibility, deferral follow-up and request matching.
- Role-based dashboards, alerts and audit logs.
- A simulator and an evaluation harness.

Non-goals

- Replacing licensed blood bank information systems or clinical decision-making.
- Crossmatching, testing or release of blood; these stay with the laboratory.
- Real patient data, payments for donations, or arranging donations outside a licensed facility.
- Live integration with real hospitals; the prototype runs on synthetic data.

SECTION 4

Users and roles

Seven roles use the platform, each with the minimum access its job needs.

ROLE	WHAT THEY DO	CAN SEE
Blood bank manager	Approves transfers, monitors regional stock and expiry risk	Stock and movements across their network
Hospital transfusion lead	Records local stock, raises requests, receives alerts	Their own hospital in full; other sites in summary
Donor coordinator	Plans drives, runs reminders, follows up deferred donors	Donor eligibility and availability, not medical details
Donor	Registers, sets availability, accepts or declines requests	Their own profile and history
Requester (family member or hospital staff)	Posts a verified blood request and tracks its status	Their own requests and match status
Administrator	Configures rules, roles and thresholds	System settings and audit logs
Auditor	Reviews who did what and when	Read-only audit trail

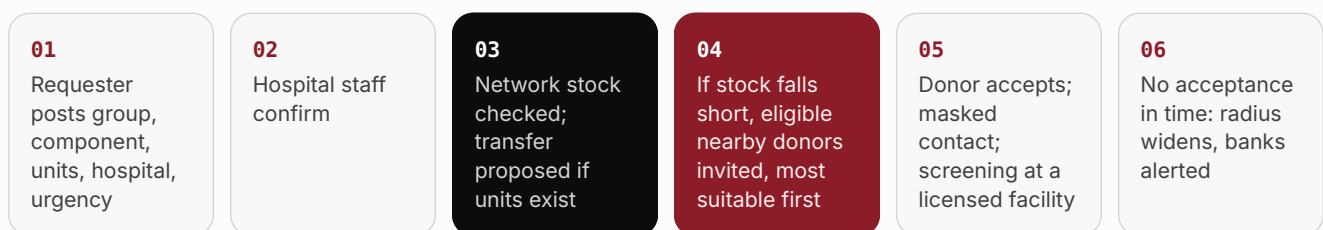
SECTION 5

What the system does

The platform is organized into eight modules that share one database and one event stream.

MODULE	WHAT IT DOES
Inventory	Tracks every unit by blood group, component, expiry, site and status (available, reserved, in transit, expired, discarded)
Compatibility	Applies ABO and Rh rules and ranks allowed substitutes, preferring exact matches and protecting scarce types such as O negative
Forecasting	Predicts daily demand per site, component and blood group, with a confidence range
Redistribution	Recommends transfers that minimize expiry waste and shortages under distance, cost and cold-chain limits; a manager approves each one
Alerts	Warns about units nearing expiry, projected stockouts and temperature excursions
Requests and matching	Verifies a blood request, checks network stock first, and only then invites eligible, opted-in donors nearby
Donor management	Registration, phone verification, eligibility and cooldown tracking, deferral follow-up, reminders, drive planning
Reporting and audit	Role-based dashboards, performance metrics and a tamper-evident log of every action

How a request flows



- 1 A requester posts a request with blood group, component, units, hospital and urgency.
- 2 Staff at the named hospital confirm it.
- 3 The system checks network stock and proposes a transfer if units exist.
- 4 If stock cannot cover it, matching invites eligible nearby donors, most suitable first.
- 5 A donor accepts; contact is masked, and the donor attends a licensed facility for screening and donation.
- 6 If nobody accepts in time, the search radius widens and blood banks are alerted.

SECTION 6

Functional requirements

The system must meet 29 functional requirements; 21 are essential (Must), 7 are important (Should) and 1 is optional (Could).

21 Must	7 Should	1 Could	29 Total
------------	-------------	------------	-------------

ID	REQUIREMENT	PRIORITY
FR-01	Record each unit with blood group, component, collection date, expiry, site and status	Must
FR-02	Log every status change and movement with time and user	Must
FR-03	Show live stock per site, group and component, with days to expiry	Must
FR-04	Reserve units for a request and release them automatically on timeout	Should
FR-05	Suggest compatible substitutes ranked by preference and rarity	Must
FR-06	Forecast daily demand per site, component and group for the next 7 days	Must
FR-07	Show forecast error against actual use	Should
FR-08	Recommend transfers that reduce projected expiry and stockouts	Must
FR-09	Require manager approval before any transfer is executed	Must
FR-10	Track transfers in transit, including temperature readings	Should
FR-11	Alert when units expire within a configurable window	Must
FR-12	Alert on projected stockout for any group or component	Must
FR-13	Alert on out-of-range storage or transport temperature	Should
FR-14	Let a verified requester create a blood request	Must
FR-15	Require confirmation by staff of the named hospital	Must
FR-16	Check network stock before contacting any donor	Must
FR-17	Rank eligible donors by distance, availability and reliability	Must
FR-18	Reveal contact details only after a donor accepts, through masked contact	Must

ID	REQUIREMENT	PRIORITY
FR-19	Widen the search radius and notify blood banks if no donor accepts in time	Should
FR-20	Register donors with phone verification and recorded consent	Must
FR-21	Enforce cooldown and eligibility rules, configurable per country	Must
FR-22	Let donors set availability and quiet hours, and delete their data	Must
FR-23	Remind deferred donors when they become eligible again	Should
FR-24	Recommend drive locations and dates from forecast gaps	Could
FR-25	Enforce role-based access for the seven roles	Must
FR-26	Keep a tamper-evident audit log of sensitive actions	Must
FR-27	Provide a dashboard per role with exportable reports	Should
FR-28	Generate synthetic sites, donors, demand and disruptions from a fixed random seed	Must
FR-29	Run policy comparisons and export the metrics	Must

Non-functional requirements

Design goals, not results

The targets below are design goals for the prototype, chosen to be testable, not measured results.

AREA	REQUIREMENT	TARGET
Performance	Dashboard pages load quickly at prototype scale (20 sites, 50,000 units)	95% of loads under 2 s
Performance	Daily redistribution plan for the whole network	Computed in under 60 s
Correctness	A unit can never be in two states or two places at once	Enforced by database transactions and constraints
Reliability	Repeated or retried transfer requests do not duplicate movements	Idempotent operations
Availability	Core inventory and alerts keep working if forecasting or matching is down	Modules fail independently
Security	Encrypted transport, encrypted personal data at rest, role-based access, phone verification	No unauthenticated access to personal data
Privacy	No public donor directory; collect only what matching needs; deletion on request	Deletion completed within 30 days
Auditability	Append-only log of sensitive actions with time, user and reason	100% of transfers, approvals and data access logged
Scalability	Add sites without redesign	Up to 100 sites in load tests
Usability	Donor and requester screens work on a phone; text is translation-ready	Core tasks in 3 taps or fewer where possible
Accessibility	Follow basic WCAG 2.1 AA practice (contrast, labels, keyboard use)	Checked with an automated audit
Reproducibility	Every simulation run can be repeated exactly	Fixed random seeds, versioned configuration
Maintainability	Core rules (compatibility, eligibility, expiry) are unit-tested	Coverage of these modules at 80% or higher

- **External services behind one interface.** SMS and email sit behind the notification module, so a provider can be swapped without touching the rest.

SECTION 8

Technology stack and why

The stack uses Python for the backend and data science, PostgreSQL for correctness, and React for the interfaces. The architecture depends on the role each tool plays (transactional store, queue, solver), so any single tool can be swapped.

LAYER	CHOICE	WHY	ALTERNATIVE AND WHY NOT
Backend API	Python with FastAPI	One language for the API, forecasting and optimization; typed request validation; automatic API documentation	Node.js or Django: fine for the API, but forecasting and solver libraries live in Python
Database	PostgreSQL with PostGIS	ACID transactions and constraints keep each unit in one state; PostGIS answers distance queries for transfers and donor matching	MongoDB: weaker guarantees for inventory state; joins and geo queries are harder to keep correct
Queue and cache	Redis with a Python job runner (RQ or Celery)	Simple to run; also holds cache and rate-limit counters	RabbitMQ or Kafka: more capable, but heavier than one team needs
Optimization	Google OR-Tools (or PuLP with CBC)	Free mixed-integer solvers with a Python interface; results can be compared against simple rule baselines	Hand-written heuristics only: quicker, but no optimality reference to measure against
Forecasting	statsmodels and scikit-learn	Seasonal time-series models are explainable and easy to evaluate before trying anything complex	Deep learning: needs far more data than a simulator gives, and is harder to explain to clinicians
Web front end	React with TypeScript (built with Vite)	Component model suits many role-based dashboards; types catch interface errors early	Server-rendered pages: simpler, but weaker for live dashboards
Maps	Leaflet with OpenStreetMap tiles	Free and lightweight; enough for site markers, transfers and donor areas	Commercial map APIs: cost and key management with no added value here
Donor and requester app	Progressive web app (installable web app)	One codebase works on any phone without an app store	Native mobile apps: double the work for the same features
Authentication	Phone one-time codes plus server-enforced roles	Donors have phones more reliably than email; roles are checked on the server, never only in the interface	Email and password only: weaker verification for donors

LAYER	CHOICE	WHY	ALTERNATIVE AND WHY NOT
Notifications	SMS and email provider behind one interface (mocked in development)	Swappable provider; no real messages sent during testing	Direct calls scattered through the code: hard to change and test
Simulation	Python with NumPy and seeded random generators	Same language as the rest; fixed seeds make experiments repeatable	Ad hoc scripts: results cannot be reproduced
Testing	pytest, plus Playwright for main end-to-end flows	Rule modules (compatibility, eligibility, expiry) are easy to unit-test to the 80% target	Manual testing only: regressions go unnoticed
Delivery	Docker Compose and GitHub Actions	The whole system starts with one command, and tests run on every change	Kubernetes: unnecessary complexity at prototype scale